

Morphological Image Processing

Elio Scaramuzzino — Mat. 7143191

elio.scaramuzzino@edu.unifi.it

Corso di Laurea Magistrale in Ingegneria Informatica

Università degli Studi di Firenze — 14/04/2026

Questo lavoro presenta un percorso di ottimizzazione progressiva delle operazioni di morfologia matematica (erosione, dilatazione, apertura, chiusura) su immagini binarie in formato PBM. Partendo da un'implementazione base, vengono introdotte ottimizzazioni sequenziali (offset linearizzati, decomposizione separabile) e una strategia originale di bit-packing che codifica l'immagine in formato byte (8 pixel/byte) e uint64 (64 pixel/word), permettendo di processare molteplici pixel con una singola operazione bitwise. Le versioni parallele CUDA sfruttano shared memory, constant memory e il bit-packing anche su GPU. I risultati mostrano speedup sequenziali fino a 16x (uint64 vs base) e speedup CUDA fino a oltre 24.000x per operazioni composte con SE di grandi dimensioni.

1. Introduzione

La morfologia matematica è una tecnica fondamentale nell'elaborazione delle immagini, utilizzata per estrarre strutture geometriche attraverso operazioni basate su un *elemento strutturante* (SE). Le quattro operazioni fondamentali — erosione, dilatazione, apertura e chiusura — trovano applicazione in compiti come la rimozione del rumore, la segmentazione e il rilevamento dei contorni.

Su immagini di grandi dimensioni e con SE di dimensioni elevate, il costo computazionale diventa significativo: la versione base richiede la scansione dell'intero SE per ogni pixel, con complessità $O(N \times M \times W_{SE} \times H_{SE})$. Questo lavoro affronta il problema con un approccio incrementale, introducendo ottimizzazioni sia algoritmiche che architetturali, culminando in kernel CUDA.

Un **contributo originale** di questo progetto è l'idea di sfruttare la natura intrinsecamente binaria delle immagini PBM per codificarle in formati compatti. In un'immagine binaria, ogni pixel è 0 o 1, eppure la rappresentazione standard spreca un intero byte (8 bit) per memorizzare un singolo bit di informazione. L'intuizione è che impacchettando i pixel — 8 in un **byte** e 64 in un **uint64_t** — le operazioni morfologiche possono essere riformulate come operazioni bitwise (AND per l'erosione, OR per la dilatazione), processando così 8 o 64 pixel *in un unico ciclo di clock*. Questo crea un parallelismo intrinseco a livello di istruzione (ILP) che si sovrappone e moltiplica con il parallelismo CUDA, portando a risultati di accelerazione eccezionali.

2. Background Teorico

2.1 Operazioni Morfologiche

L'**erosione** verifica, per ogni pixel, se l'intero SE è contenuto nel vicinato: se tutti i pixel coperti dall'SE sono a 1, il pixel di output è 1, altrimenti 0. La **dilatazione** opera dualmente: se almeno un pixel del vicinato è a 1, il pixel di output è 1. **Apertura** (erosione + dilatazione) e **chiusura** (dilatazione + erosione) sono operazioni composte che richiedono due passate sull'immagine.

2.2 Elemento Strutturante

L'SE definisce la forma e la dimensione del vicinato. In questo progetto sono supportati SE di tipo **box** (quadrato pieno, separabile) e **disk** (approssimazione circolare, non separabile). Le dimensioni testate vanno da 3×3 fino a 37×37 .

3. Implementazioni Sequenziali

3.1 Versione Base

L'implementazione base itera su ogni pixel dell'immagine e scandisce l'intero SE con un doppio ciclo annidato (riga, colonna del kernel). Per ogni posizione con valore 1, verifica il pixel corrispondente nell'immagine. La complessità è $O(N \times M \times W_{SE} \times H_{SE})$ ed è indipendente dal contenuto effettivo del kernel: anche i pixel a 0 dell'SE vengono esaminati.

3.2 Versione con Offset Linearizzati

La prima ottimizzazione pre-calcola gli *offset* dei soli pixel attivi (valore 1) dell'SE, riducendo i cicli interni da 4 a 3. Gli offset vengono linearizzati ($\text{offset} = dy \times W + dx$), trasformando l'accesso in memoria da un calcolo bidimensionale a una semplice somma. Lo speedup tipico è di circa **1.5x** per il box SE e cresce per SE con bassa densità come il disk.

3.3 Decomposizione Separabile

Per SE di tipo box, l'erosione con un kernel $N \times N$ equivale a un'erosione orizzontale (linea $1 \times N$) seguita da una verticale (linea $N \times 1$). La complessità scende da $O(N^2)$ a $O(2N)$ per pixel. Il beneficio è particolarmente evidente per le operazioni composte con SE grandi: per il closing con SE 37×37 , lo speedup rispetto alla base raggiunge **35.5x**. Non applicabile al disk SE.

3.4 Bit-Packing: l'Idea Chiave

L'idea centrale di questo progetto nasce dall'osservazione che in un'immagine binaria PBM ogni pixel è un singolo bit, ma la rappresentazione standard alloca un **intero byte** per pixel — uno spreco di 7 bit su 8. La domanda è: *è possibile sfruttare questi bit inutilizzati per processare più pixel simultaneamente?*

La risposta è sì, e la chiave è la **riformulazione delle operazioni morfologiche in termini di operazioni bitwise**. Impacchettando i pixel in modo che ogni byte contenga 8 pixel consecutivi (e ogni uint64_t ne contenga 64), le operazioni diventano:

- **Erosione:** l'operazione "tutti i vicini sono 1" diventa un AND bitwise tra l'accumulatore e il byte/word corrispondente a ciascun offset dell'SE. Se l'offset dx non è multiplo di 8 (o 64), si leggono due byte/word adiacenti e si compone il valore corretto tramite shift-left e shift-right combinati. Se l'accumulatore diventa zero, si interrompe il ciclo (*early exit*): tutti gli 8 o 64 pixel sono

già determinati.

- **Dilatazione:** dualmente, l'operazione "almeno un vicino è 1" diventa un OR con scatter del valore sorgente su tutti gli offset.

Il vantaggio è triplice: (1) ogni istruzione AND/OR processa **8 pixel** (byte) o **64 pixel** (uint64) contemporaneamente, ottenendo un parallelismo a livello di istruzione intrinseco; (2) il footprint in memoria si riduce di 8x o 64x, migliorando drammaticamente la località della cache (l'intera immagine "pesa" 64 volte meno); (3) l'early exit su un accumulatore a 64 bit termina simultaneamente la computazione per 64 pixel quando sono tutti zero.

Lo speedup sequenziale misurato è di **3-4x** per byte e **14-16x** per uint64 sull'erosione. Questo supera significativamente il fattore 8x che ci si aspetterebbe dal solo rapporto di compressione, grazie al contributo della migliore località di cache.

4. Implementazioni CUDA

4.1 Architettura Generale

Tutte le versioni CUDA condividono la stessa architettura host-side: l'SE viene copiato nella **constant memory** (array di int2 con offset dx, dy), le immagini vengono allocate nella global memory, e il kernel viene lanciato con una griglia 2D di blocchi.

4.2 Kernel Naive (Global Memory)

Ogni thread gestisce un pixel. Per l'erosione, il thread scorre gli offset dell'SE nella constant memory e accede direttamente alla global memory. Nonostante gli accessi non coalescenti, la cache L1/L2 della GPU fornisce un buon livello di caching.

4.3 Kernel con Shared Memory

Il kernel ottimizzato carica cooperativamente un *tile* in shared memory, includendo un alone (halo) pari ai margini dell'SE. La computazione avviene sulla shared memory, eliminando accessi ridondanti alla global. Tuttavia, l'overhead di caricamento e sincronizzazione non sempre viene ammortizzato, come vedremo nei risultati.

4.4 Kernel Byte e Uint64 su GPU

La logica di bit-packing viene portata su CUDA. Il kernel byte opera su ByteImage dove ogni thread processa un byte = 8 pixel; il kernel uint64 opera su Uint64Image dove ogni thread processa un uint64_t = 64 pixel. Entrambi utilizzano shared memory con tile appropriati e gestione dell'halo in unità di byte/word.

Il risultato è un'**accelerazione moltiplicativa**: il parallelismo CUDA (migliaia di thread simultanei) si combina con il parallelismo a livello di dato del bit-packing (64 pixel per istruzione). Ogni thread CUDA con uint64 processa 64 pixel: con un blocco 32x8 = 256 thread, un singolo blocco copre 256 x 64 = 16.384 pixel.

5. Setup Sperimentale

I benchmark sono stati eseguiti su un'immagine PBM di test con i seguenti parametri: SE box (dimensioni 3, 5, 7, 9, 11, 15, 21, 37) e disk (raggio 1-18); operazioni: erosione, dilatazione, apertura, chiusura; configurazioni di blocco CUDA: 8x8, 16x16, 32x8, 32x16, 32x32 per i kernel standard e 32x4, 32x8, 64x4, 64x8 per il kernel uint64.

6. Risultati

6.1 Confronto Versioni Sequenziali

6.1.1 Erosione

La Tabella I riporta i tempi dell'erosione sequenziale con SE box.

Table I: Erosione sequenziale (Box SE) — tempi in ms.

SE	Base	Offset	Sep.	Byte	Uint64
3	305.6	202.5	247.9	90.23	19.30
5	280.1	178.8	221.3	88.74	19.80
7	275.1	173.1	211.8	86.17	19.13
9	274.7	173.1	207.0	91.49	20.78
11	273.6	178.0	212.7	92.95	20.09
15	273.5	172.1	212.9	94.54	19.10
21	270.3	174.1	220.7	88.50	19.75
37	271.7	172.0	212.7	85.32	19.34

Un dato immediatamente evidente è che i tempi dell'erosione base sono sostanzialmente costanti al variare della dimensione dell'SE (~271-306 ms). Questo perché l'immagine di test ha una struttura che porta all'*early exit* del ciclo interno indipendentemente dalla dimensione del kernel: appena si trova un pixel a 0 sotto l'SE, si interrompe la verifica. Lo stesso vale per offset, byte e uint64.

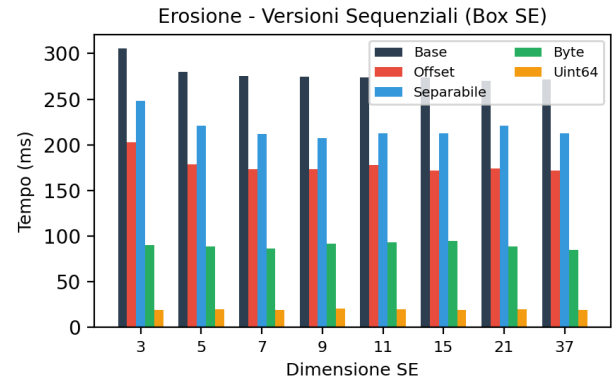


Figure 1: Tempi dell'erosione sequenziale per tutte le dimensioni del SE (Box).

6.1.2 Dilatazione

Table II: Dilatazione sequenziale (Box SE) — tempi in ms.

SE	Base	Offset	Sep.	Byte	Uint64
3	534.8	261.3	307.0	145.0	19.57
5	1211	495.9	359.4	399.6	54.02
7	2181	905.1	449.2	789.0	110.5
9	3520	1507	540.5	1334	187.3
11	5319	2232	626.7	2029	284.0
15	9626	3983	826.7	3769	527.2
21	18881	7908	1128	7387	1060
37	56368	23753	1990	22205	3167

A differenza dell'erosione, la dilatazione mostra un costo base che **cresce quadraticamente** con la dimensione dell'SE: da 535 ms (SE 3) a oltre 56 secondi (SE 37). La ragione è strutturale: nella dilatazione, per ogni pixel sorgente a 1, si "stampa" l'intero SE sull'output. Non esiste un meccanismo di early exit analogo all'erosione, perché ogni pixel attivo deve propagare il suo contributo a tutti gli offset. Il costo è quindi proporzionale al numero di pixel a 1 moltiplicato per l'area dell'SE.

La versione separabile è particolarmente efficace qui: per SE 37, riduce il tempo da 56.4 secondi a soli 2.0 secondi (28.3x). Anche byte e uint64 seguono la crescita quadratica, poiché il bit-packing accelera la singola operazione ma non cambia la complessità algoritmica. L'uint64 offre comunque un'accelerazione notevole: da 56.4 secondi (base) a 3.2 secondi (17.8x) per SE 37.

6.1.3 Apertura

Table III: Apertura sequenziale (Box SE) — tempi in ms.

SE	Base	Offset	Sep.	Byte	Uint64
3	345.0	244.1	324.9	99.03	22.82
5	315.9	212.4	277.5	91.69	21.85
7	321.4	210.6	273.1	92.03	19.98
9	305.2	210.8	275.2	95.26	19.97
11	313.5	209.7	278.8	101.1	20.10
15	309.5	207.1	272.6	95.66	19.57
21	313.8	215.1	285.4	96.24	21.03
37	306.1	211.2	274.3	93.11	19.56

L'apertura (erosione + dilatazione) presenta tempi base relativamente stabili (~306-345 ms), simili all'erosione. Questo perché la prima passata (erosione) è rapida grazie all'early exit, e la seconda passata (dilatazione) opera su un'immagine già erosa con pochi pixel a 1, quindi anche la dilatazione risulta veloce. Il costo non dipende significativamente dalla dimensione dell'SE, poiché l'erosione "svuota" l'immagine e la dilatazione successiva ha poco lavoro. L'uint64 offre uno speedup costante di ~15x.

6.1.4 Chiusura

Table IV: Chiusura sequenziale (Box SE) — tempi in ms.

SE	Base	Offset	Sep.	Byte	Uint64
3	1189	562.6	519.4	270.3	40.05
5	2954	1305	694.9	763.9	117.5
7	6179	2572	888.2	1524	223.5
9	9318	4231	1079	2571	376.6
11	13729	6176	1271	3968	562.8
15	25299	11287	1695	7293	1051
21	52215	21704	2583	13909	2019
37	153992	66430	4336	42381	6205

La chiusura (dilatazione + erosione) è l'operazione più costosa. La prima passata (dilatazione) ha costo proporzionale all'area dell'SE, e la seconda (erosione) opera su un'immagine dilatata con molti pixel a 1, quindi anche l'erosione diventa costosa (l'early exit è meno frequente). Il tempo base cresce da 1.2 secondi (SE 3) a 154 secondi (SE 37).

La versione separabile è qui dominante: per SE 37, riduce il tempo da 154 secondi a soli 4.3 secondi (35.5x). La separabile supera l'uint64 (6.2 secondi) per SE grandi: la riduzione di complessità algoritmica (da $O(N^2)$ a $O(2N)$) domina sulla riduzione del lavoro per istruzione offerta dal bit-packing.

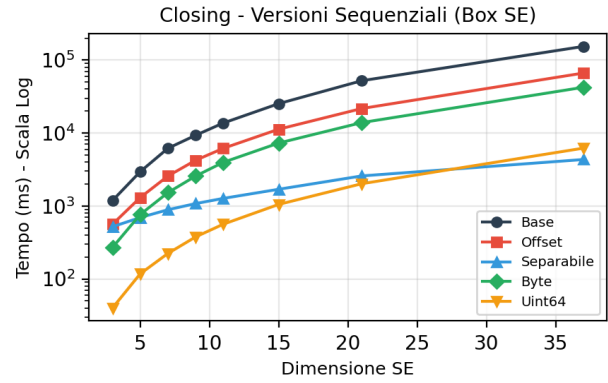


Figure 2: Chiusura sequenziale in scala logaritmica. La separabile cresce linearmente.

6.1.5 Tabelle Riassuntive Speedup Sequenziali

Table 5: Speedup sequenziali (Erosione) vs Base.

SE	Off/Base	Sep/Base	Byte/Base	U64/Base
3	1.5x	1.2x	3.4x	15.8x
5	1.6x	1.3x	3.2x	14.1x
7	1.6x	1.3x	3.2x	14.4x
9	1.6x	1.3x	3.0x	13.2x
11	1.5x	1.3x	2.9x	13.6x
15	1.6x	1.3x	2.9x	14.3x
21	1.6x	1.2x	3.1x	13.7x
37	1.6x	1.3x	3.2x	14.1x

Table 6: Speedup sequenziali (Dilatazione) vs Base.

SE	Off/Base	Sep/Base	Byte/Base	U64/Base
3	2.0x	1.7x	3.7x	27.3x
5	2.4x	3.4x	3.0x	22.4x
7	2.4x	4.9x	2.8x	19.7x
9	2.3x	6.5x	2.6x	18.8x
11	2.4x	8.5x	2.6x	18.7x
15	2.4x	11.6x	2.6x	18.3x
21	2.4x	16.7x	2.6x	17.8x
37	2.4x	28.3x	2.5x	17.8x

Table 7: Speedup sequenziali (Apertura) vs Base.

SE	Off/Base	Sep/Base	Byte/Base	U64/Base
3	1.4x	1.1x	3.5x	15.1x
5	1.5x	1.1x	3.4x	14.5x
7	1.5x	1.2x	3.5x	16.1x
9	1.4x	1.1x	3.2x	15.3x
11	1.5x	1.1x	3.1x	15.6x
15	1.5x	1.1x	3.2x	15.8x
21	1.5x	1.1x	3.3x	14.9x
37	1.4x	1.1x	3.3x	15.6x

Table 8: Speedup sequenziali (Chiusura) vs Base.

SE	Off/Base	Sep/Base	Byte/Base	U64/Base
3	2.1x	2.3x	4.4x	29.7x
5	2.3x	4.3x	3.9x	25.1x
7	2.4x	7.0x	4.1x	27.6x
9	2.2x	8.6x	3.6x	24.7x
11	2.2x	10.8x	3.5x	24.4x
15	2.2x	14.9x	3.5x	24.1x
21	2.4x	20.2x	3.8x	25.9x
37	2.3x	35.5x	3.6x	24.8x

Le tabelle confermano pattern distinti per ogni operazione. Per **erosione** e **apertura**, gli speedup sono stabili al variare dell'SE (uint64 ~15x). Per la **dilatazione**, lo speedup della separabile cresce con l'SE (fino a 28.3x) e domina sulle altre

versioni. Per la **chiusura**, la separabile raggiunge il massimo speedup (35.5x) grazie alla doppia riduzione di complessità su entrambe le passate. L'uint64 offre sempre speedup significativi (14-30x) indipendentemente dall'operazione, confermando la robustezza dell'approccio bit-packing.

6.2 Analisi Block Size CUDA

La Tabella IX riporta i tempi medi del kernel di erosione per diverse configurazioni di blocco CUDA.

Table IX: Tempi medi erosione CUDA (ms) per block size.

Block	Offset	Naive	Byte
8x8	4.479	1.588	0.5682
16x16	2.790	1.567	0.5095
32x8	2.842	1.677	0.5291
32x16	2.587	1.667	0.5037
32x32	2.650	1.847	0.4978

Il kernel naive ottiene le migliori prestazioni con blocco **16×16** (1.567 ms), il kernel offset/shared con **32×16** (2.587 ms), il byte con **32×32** (0.498 ms). Per il kernel uint64, il blocco migliore è **32×8** (0.092 ms).

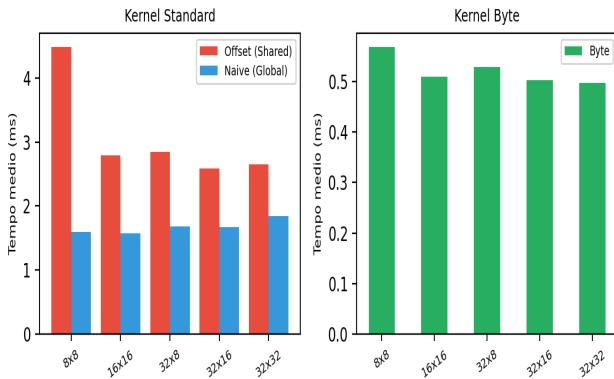


Figure 3: Tempi medi di erosione per diverse configurazioni di blocco CUDA.

Un risultato sorprendente è che il kernel naive supera il kernel con shared memory. La spiegazione è triplice: (1) per SE piccoli il riuso dei dati nel tile è limitato e l'overhead di caricamento e sincronizzazione non viene ammortizzato; (2) la cache L1/L2 della GPU fornisce già un buon caching per accessi con buona località spaziale; (3) il blocco di shared memory richiede un halo crescente con l'SE, aumentando il volume di dati da caricare cooperativamente.

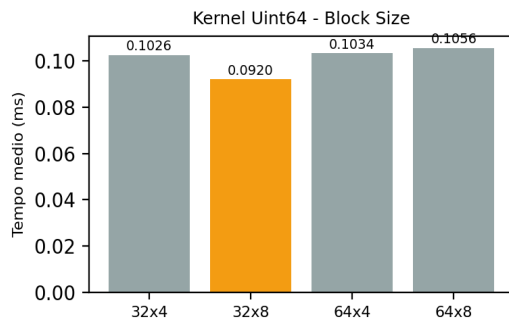


Figure 4: Tempi del kernel Uint64 per blocco. 32x8 è il migliore.

Per il kernel uint64, 32×8 è ottimale. Con 256 thread (vs 1024 di 32×32), si limita l'uso di shared memory per tile e ogni thread processa 64 pixel, coprendo un'ampia porzione dell'immagine anche con pochi thread per blocco.

6.3 Confronto Globale

Table X: Chiusura — confronto finale (ms, best block size).

SE	Base	Sep.	U64	C.Off.	C.Byte	C.U64
3	1189	519.4	40.05	4.863	0.4270	0.0753
5	2954	694.9	117.5	6.938	0.8677	0.1374
7	6179	888.2	223.5	14.25	1.684	0.3363
9	9318	1079	376.6	16.32	2.412	0.3881
11	13729	1271	562.8	22.14	3.527	0.5664
15	25299	1695	1051	39.30	6.406	1.264
21	52215	2583	2019	65.69	12.43	2.009
37	153992	4336	6205	189.2	38.58	6.323

Per il closing con SE 37×37, il kernel CUDA uint64 impiega **6.32 ms** rispetto ai **154 secondi** della versione base: uno speedup di circa **24.353x**. Questo combina tre livelli: parallelismo CUDA, bit-packing e shared memory.

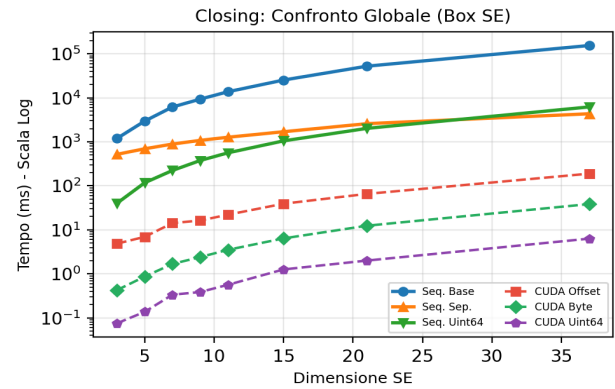


Figure 5: Confronto globale per il closing. Si noti la distanza di ordini di grandezza.

6.4 Analisi per Operazione

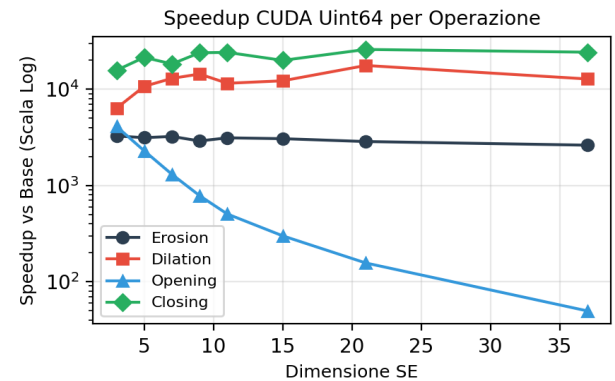


Figure 6: Speedup CUDA Uint64 rispetto alla base, per operazione.

Per **erosione**, lo speedup è relativamente costante (~3000x) perché il costo sequenziale non cresce con la dimensione dell'SE (grazie all'early exit). Per la **dilatazione**, lo speedup cresce con l'SE perché la versione base ha un costo proporzionale all'area dell'SE (ogni pixel sorgente a 1 "stampa" tutto l'SE), raggiungendo oltre 12.000x per SE 37×37.

Per la **chiusura**, lo speedup cresce rapidamente con la dimensione dell'SE perché questa operazione composta ha un costo base che scala quadraticamente, raggiungendo il massimo di oltre 24.000x per SE 37×37.

L'**apertura** mostra un comportamento opposto: lo speedup *decresce* all'aumentare della dimensione dell'SE. Questo è il risultato più controintuitivo del benchmark. La ragione è che

l'apertura (erosione + dilatazione) ha un costo sequenziale che rimane basso per l'erosione (grazie all'early exit), mentre la seconda passata (dilatazione) su un'immagine già erosa è rapida (pochi pixel a 1). Sul lato CUDA, invece, il costo dei due lanci kernel consecutivi con grandi tile di shared memory cresce significativamente. Per SE 37×37, il kernel CUDA offset diventa addirittura più lento della versione sequenziale (speedup ~0.9x), mentre l'uint64 mantiene comunque ~50x.

6.5 Tabelle Riassuntive Speedup CUDA

Table 11: Speedup CUDA (Erosione) vs Base seq.

SE	C.Off.	C.Naive	C.Byte	C.U64
3	114x	149x	521x	3274x
5	138x	177x	599x	3142x
7	131x	174x	585x	3237x
9	117x	170x	535x	2905x
11	120x	194x	583x	3131x
15	111x	186x	582x	3069x
21	98.7x	193x	547x	2865x
37	66.9x	191x	532x	2627x

Table 12: Speedup CUDA (Dilatazione) vs Base seq.

SE	C.Off.	C.Naive	C.Byte	C.U64
3	198x	160x	978x	6364x
5	566x	535x	2479x	10749x
7	998x	973x	4374x	13004x
9	1558x	1557x	7176x	14432x
11	1539x	1670x	7895x	11563x
15	2757x	2991x	14072x	12272x
21	6637x	7660x	35572x	17714x
37	9197x	15611x	72717x	12825x

Table 13: Speedup CUDA (Apertura) vs Base seq.

SE	C.Off.	C.Naive	C.Byte	C.U64
3	63.0x	64.3x	706x	4108x
5	46.5x	39.9x	345x	2294x
7	29.8x	24.1x	197x	1307x
9	19.2x	14.7x	119x	784x
11	14.3x	9.5x	82.7x	505x
15	8.6x	5.2x	45.2x	300x
21	4.9x	3.0x	23.5x	156x
37	1.6x	0.9x	7.5x	49.6x

Table 14: Speedup CUDA (Chiusura) vs Base seq.

SE	C.Off.	C.Naive	C.Byte	C.U64
3	245x	227x	2785x	15801x
5	426x	410x	3405x	21496x
7	433x	412x	3669x	18371x
9	571x	570x	3864x	24009x
11	620x	604x	3893x	24237x
15	644x	536x	3950x	20018x
21	795x	663x	4202x	25996x
37	814x	711x	3992x	24353x

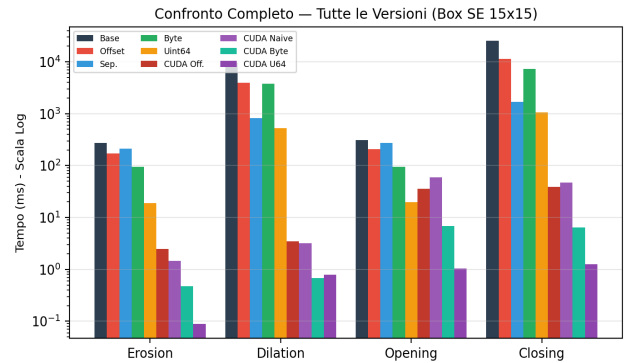


Figure 7: Confronto completo di tutte le versioni per Box SE 15x15.

6.6 Scelta della Versione Ottimale

I risultati mostrano che non esiste un'unica versione "migliore" in assoluto: la scelta dipende dal contesto d'uso e dai vincoli.

Se non si dispone di GPU, la scelta dipende dalla forma dell'SE e dalla dimensione. Per SE box di grandi dimensioni e operazioni composte, la **versione separabile** offre il miglior rapporto tra complessità implementativa e prestazioni (fino a 35.5x), con il limite di non essere applicabile al disk SE. Per SE di qualsiasi forma e dimensione, la **versione uint64** offre uno speedup consistente di 14-16x con un costo implementativo moderato (gestione dello shift per gli offset non allineati). La versione byte rappresenta un buon compromesso intermedio (3-4x), più semplice da implementare del uint64 poiché i casi di shift coinvolgono solo 2 byte anziché 2 word.

Se si dispone di GPU, la versione **CUDA uint64** è la scelta ottimale per la quasi totalità degli scenari, con speedup che vanno da ~3.000x a oltre 24.000x. L'unica eccezione è l'apertura con SE molto grandi (>37), dove il costo dei due lanci kernel consecutivi con grandi tile riduce il vantaggio. In questo caso, la versione CUDA byte offre un compromesso migliore (7.5x vs 50x dell'uint64 — comunque vantaggiosa rispetto al sequenziale). Il kernel naive è preferibile al kernel con shared memory per la sua semplicità implementativa e prestazioni comparabili o superiori.

In sintesi, la **strategia raccomandata** è: uint64 come rappresentazione dati per tutte le immagini binarie (il costo di conversione è trascurabile), decomposizione separabile dove applicabile su CPU, e CUDA uint64 come kernel GPU di default con fallback a CUDA byte per SE di grandi dimensioni nell'apertura.

7. Analisi del Disk SE

L'elemento strutturante circolare presenta caratteristiche diverse dal box. La Tabella VIII riporta l'erosione con disk SE di raggio crescente.

Table XV: Erosione con Disk SE — tempi in ms.

Raggio	Base	Offset	Byte	Uint64
1	403.5	233.4	72.40	12.34
2	464.2	264.8	94.23	23.26
3	509.4	230.7	84.04	20.32
4	557.0	238.6	91.32	20.28
5	595.1	239.7	92.24	20.19
7	720.4	246.1	95.76	20.30
10	890.7	239.9	85.33	20.16
18	1431	237.0	86.18	20.55

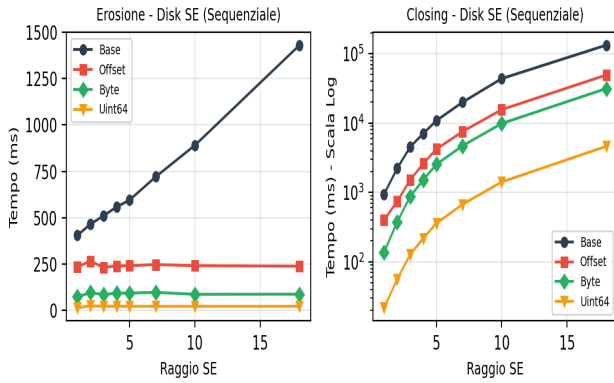


Figure 8: Tempi sequenziali per Disk SE: erosione (lineare) e closing (log).

Un aspetto notevole è che per il disk SE la versione base cresce significativamente con il raggio (da 404 ms a 1431 ms), mentre le versioni offset, byte e uint64 rimangono sostanzialmente costanti. La spiegazione risiede nel comportamento dell'early exit: la versione base itera sull'intera matrice del kernel (inclusi i pixel a 0), il cui costo cresce con l'area del bounding box. Le versioni ottimizzate iterano solo sui pixel attivi e terminano appena trovano un mismatch. Il numero di pixel attivi lungo il perimetro che causano l'early exit rimane simile indipendentemente dal raggio.

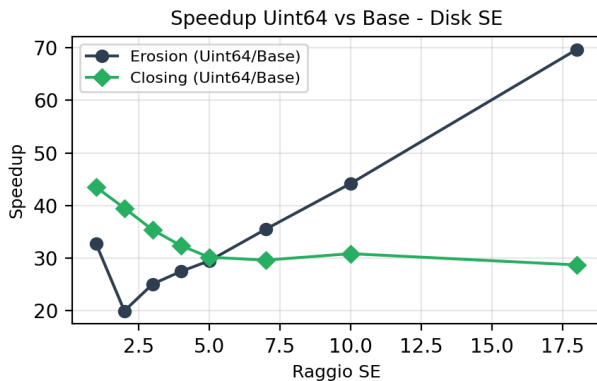


Figure 9: Speedup Uint64 vs Base per il Disk SE.

La decomposizione separabile **non è applicabile** al disk SE, rendendo le versioni byte e uint64 le uniche alternative efficaci per SE circolari di grandi dimensioni. Per il closing con disk SE di raggio 18, lo speedup uint64/base raggiunge **28.7x**.

8. Conclusioni

Questo lavoro ha dimostrato come un approccio incrementale all'ottimizzazione possa portare a miglioramenti delle prestazioni di diversi ordini di grandezza. La progressione dei risultati segue una chiara gerarchia: le ottimizzazioni algoritmiche (offset, separabile) offrono speedup moderati ma generali; il bit-packing sfrutta la specificità delle immagini binarie per ottenere un parallelismo a livello di dato; CUDA aggiunge il parallelismo massivo della GPU. La combinazione di tutti e tre i livelli — come nel kernel CUDA uint64 — porta a speedup cumulativi dell'ordine di 3.000-24.000x.

L'analisi ha anche evidenziato risultati non banali: la shared memory non è sempre vantaggiosa rispetto alla global memory con cache; la decomposizione separabile può superare il bit-packing per operazioni composte con SE grandi; l'apertura

mostra uno speedup CUDA decrescente con l'SE a differenza della chiusura; e il comportamento dell'early exit gioca un ruolo cruciale. Questi risultati sottolineano l'importanza di un'analisi empirica accurata, poiché le euristiche teoriche non sempre predicono il comportamento reale su hardware moderno.